



in collaboration with



Lab 4: A Practical Approach to Understanding System Internals

Aaisma Wagle

Bsc (Hons) Computer Science with Artificial Intelligence

Softwarica College of IT and E-commerce

ST5003CMD Operating Systems, Security, and Networks

Rupak Rajbanshi

28th September, 2026

TABLE OF CONTENT

INTRODUCTION	1
LABORATORY EXERCISES	2
A. Lab 1: Data Type Size Exploration	2
B. Lab 2: Memory Segment Address Mapping	4
KNOWLEDGE CHECK QUESTIONS	6
Question 1	6
Question 2	6
Question 3	6
Question 4	7
Question 5	7
Question 6	7
CONCLUSION	7
GITHUB REPOSITORY	7

LIST OF FIGURES

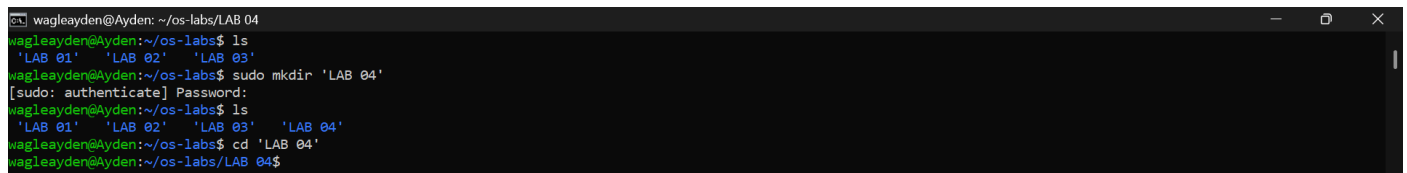
Figure 01: Make Lab 04 Folder to Work With	1
Figure 02: Make task1 Repository and lab1.c File	2
Figure 03: lab1.c File in Nano Editor	2
Figure 04: lab1.c File View using cat Command	2
Figure 05: Run GCC compilation of lab1.c	2
Figure 06: Data Type and Space Division	3
Figure 07: Make task2 Repository and lab2.c File	4
Figure 08: lab2.c File in Nano Editor	4
Figure 09: lab2.c File View using cat Command	4
Figure 10: Run GCC compilation of lab2.c	5

INTRODUCTION

This lab looks at how data types and memory management work inside an operating system. A data type decides how much memory a variable uses and how that memory is read. The operating system also divides a process's memory into segments: Text, Data, BSS, Heap, and Stack.

In Lab 1, the `sizeof()` operator is used in C to find the size of different data types on my system, and the results are compared with the table in pdf. In Lab 2, the addresses of variables stored in different segments are printed to see how the memory is arranged. Both programs were written and run in the Linux terminal using `gcc`. The aim of this lab is to understand how programs use memory in practice.

```
Sudo mkdir <repository name> //For creating a new folder.  
(Sudo is used to access files that regular user account doesn't have access to)  
  
ls //list all files in the directory  
  
sudo nano <file name> //Opens the nano text editor to write or edit a C file  
  
cat <file name> //Shows the contents of a file in the terminal  
  
cd <folder name> //Moves into a folder  
  
sudo gcc <file name>.c -o <output name> //Compiles the C file into a runnable program.  
( -o sets the name of the output file)  
  
./<output name> //Runs the compiled program
```

A terminal window screenshot showing a user named 'wagleayden' at a machine named 'Ayden'. The user is in the directory '~/os-labs/LAB 04'. They run 'ls' and see 'LAB 01', 'LAB 02', and 'LAB 03'. Then they run 'sudo mkdir 'LAB 04'', which prompts for a password. After authentication, they run 'ls' again and see 'LAB 01', 'LAB 02', 'LAB 03', and 'LAB 04'. Finally, they run 'cd 'LAB 04'' and the prompt changes to '~/os-labs/LAB 04\$'.

```
wagleayden@Ayden: ~/os-labs/LAB 04  
wagleayden@Ayden:~/os-labs$ ls  
'LAB 01' 'LAB 02' 'LAB 03'  
wagleayden@Ayden:~/os-labs$ sudo mkdir 'LAB 04'  
[sudo: authenticate] Password:  
wagleayden@Ayden:~/os-labs$ ls  
'LAB 01' 'LAB 02' 'LAB 03' 'LAB 04'  
wagleayden@Ayden:~/os-labs$ cd 'LAB 04'  
wagleayden@Ayden:~/os-labs/LAB 04$
```

Figure 01: Make Lab 04 Folder to Work With

LABORATORY EXERCISES

A. Lab 1: Data Type Size Exploration

Objective: Use `sizeof()` to observe how much memory each data type uses on your system.

Tasks:

1) Create a C program that Prints the Size of: *char, int, float, double, long, unsigned int, long long*:

```
wagleayden@Ayden: ~/os-labs/LAB 04/task1
wagleayden@Ayden:~/os-labs/LAB 04$ ls
task1 task2
wagleayden@Ayden:~/os-labs/LAB 04$ cd task1
wagleayden@Ayden:~/os-labs/LAB 04/task1$ sudo nano lab1.c
```

Figure 02: Make task1 Repository and lab1.c File

```
GNU nano 8.7.1 lab1.c *
#include <stdio.h>

int main(){
    printf("---Data Type Sizes on This System---\n");

    printf("char:   %zu bytes\n", sizeof(char));
    printf("int:    %zu bytes\n", sizeof(int));
    printf("float:  %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("long:   %zu bytes\n", sizeof(long));
    printf("pointer: %zu bytes\n", sizeof(int*));

    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long long:   %zu bytes\n", sizeof(long long));

    return 0;
}
```

Figure 03: lab1.c File in Nano Editor

```
wagleayden@Ayden:~/os-labs/LAB 04/task1$ cat lab1.c
#include <stdio.h>

int main() {
    printf("---Data Type Sizes on This System---\n");

    printf("char:   %zu bytes\n", sizeof(char));
    printf("int:    %zu bytes\n", sizeof(int));
    printf("float:  %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("long:   %zu bytes\n", sizeof(long));
    printf("pointer: %zu bytes\n", sizeof(int*));

    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long long:   %zu bytes\n", sizeof(long long));

    return 0;
}
wagleayden@Ayden:~/os-labs/LAB 04/task1$
```

Figure 04: lab1.c File View using cat Command

2) Compile and Run on your System:

```
wagleayden@Ayden: ~/os-labs/LAB 04/task1
wagleayden@Ayden:~/os-labs/LAB 04/task1$ sudo gcc lab1.c -o lab1
wagleayden@Ayden:~/os-labs/LAB 04/task1$ ./lab1
---Data Type Sizes on This System---
char:   1 bytes
int:    4 bytes
float:  4 bytes
double: 8 bytes
long:   8 bytes
pointer: 8 bytes
unsigned int: 4 bytes
long long: 8 bytes
wagleayden@Ayden:~/os-labs/LAB 04/task1$
```

Figure 05: Run GCC compilation of lab1.c

3) Compare output with Table I:

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

Figure 06: Data Type and Space Division

The output in [Figure 06](#) shows that char uses 1 byte, int and float use 4 bytes, and double uses 8 bytes. long uses 8 bytes, unsigned int uses 4 bytes, long long uses 8 bytes, and a pointer uses 8 bytes. These sizes match the 64-bit column of Table I, so my system is a 64-bit system.

Why might sizeof(long) differ between 32-bit and 64-bit systems?

The size of long depends on the system architecture. On a 32-bit system, long is 4 bytes and is the same size as the memory address on that system. On a 64-bit system, long is 8 bytes. It can store 64-bit addresses and much larger values. This is why sizeof(long) gives 4 on a 32-bit system and 8 on a 64-bit system.

B. Lab 2: Memory Segment Address Mapping

Objective: Observe where different variables live in memory and validate OS memory layout.

Tasks:

1) Write a Program that Declares Variables in each Segment:

- *Global initialized variable (Data segment)*
- *Global uninitialized variable (BSS segment)*
- *Local variable (Stack segment)*
- *Dynamically allocated variable (Heap segment)*

```
wagleayden@Ayden: ~/os-labs/LAB 04/task2
wagleayden@Ayden:~/os-labs/LAB 04$ sudo mkdir task2
wagleayden@Ayden:~/os-labs/LAB 04$ cd task2
wagleayden@Ayden:~/os-labs/LAB 04/task2$ sudo touch lab2.c
wagleayden@Ayden:~/os-labs/LAB 04/task2$ sudo nano lab2.c
```

Figure 07: Make task2 Repository and lab2.c File

```
GNU nano 8.7.1 lab2.c *
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int global_init = 100; //Data segment
int global_uninit; //BSS segment

int main() {
    int local_var = 30; //Stack
    int *heap_var = (int *)malloc(sizeof(int)); //Heap
    *heap_var = 500;

    printf("---Memory Segment Address---\n\n");

    printf("global_init: %p (Data Segment)\n", (void *)&global_init);
    printf("global_uninit: %p (BSS Segment)\n", (void *)&global_uninit);
    printf("local_var: %p (Stack Segment)\n", (void *)&local_var);
    printf("heap_var data: %p (Heap Segment)\n", (void *)heap_var);

    uintptr_t stack_addr = (uintptr_t)&local_var;
    uintptr_t heap_addr = (uintptr_t)heap_var;
    uintptr_t diff = (stack_addr > heap_addr) ? (stack_addr - heap_addr) : (heap_addr - stack_addr);

    printf("\nDifference Between Stack and Heap: %llu bytes (0x%llx)\n", (unsigned long long)diff, (unsigned long long)diff);

    free(heap_var);
    return 0;
}
```

Figure 08: lab2.c File in Nano Editor

```
wagleayden@Ayden:~/os-labs/LAB 04/task2$ cat lab2.c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int global_init = 100; //Data segment
int global_uninit; //BSS segment

int main() {
    int local_var = 30; //Stack
    int *heap_var = (int *)malloc(sizeof(int)); //Heap
    *heap_var = 500;

    printf("---Memory Segment Address---\n\n");

    printf("global_init: %p (Data Segment)\n", (void *)&global_init);
    printf("global_uninit: %p (BSS Segment)\n", (void *)&global_uninit);
    printf("local_var: %p (Stack Segment)\n", (void *)&local_var);
    printf("heap_var data: %p (Heap Segment)\n", (void *)heap_var);

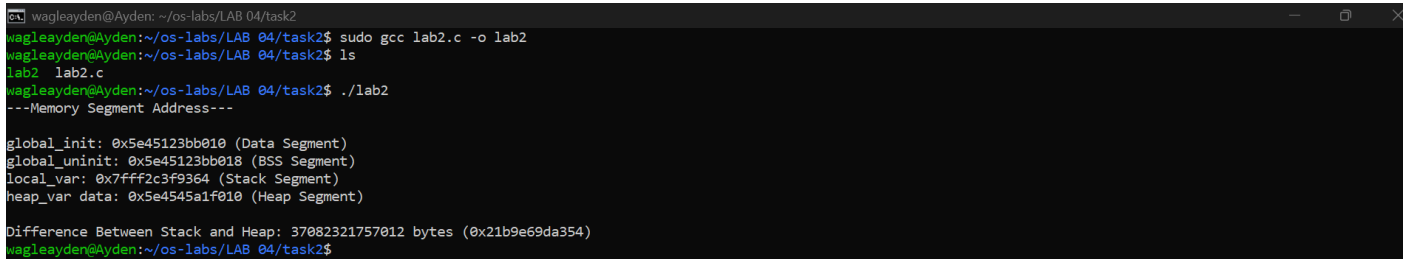
    uintptr_t stack_addr = (uintptr_t)&local_var;
    uintptr_t heap_addr = (uintptr_t)heap_var;
    uintptr_t diff = (stack_addr > heap_addr) ? (stack_addr - heap_addr) : (heap_addr - stack_addr);

    printf("\nDifference Between Stack and Heap: %llu bytes (0x%llx)\n", (unsigned long long)diff, (unsigned long long)diff);

    free(heap_var);
    return 0;
}
```

Figure 09: lab2.c File View using cat Command

2) Print the Address of each Variable using %p:



```
wagleayden@Ayden: ~/os-labs/LAB 04/task2
wagleayden@Ayden:~/os-labs/LAB 04/task2$ sudo gcc lab2.c -o lab2
wagleayden@Ayden:~/os-labs/LAB 04/task2$ ls
lab2  lab2.c
wagleayden@Ayden:~/os-labs/LAB 04/task2$ ./lab2
---Memory Segment Address---
global_init: 0x5e45123bb010 (Data Segment)
global_uninit: 0x5e45123bb018 (BSS Segment)
local_var: 0x7fff2c3f9364 (Stack Segment)
heap_var data: 0x5e4545a1f010 (Heap Segment)
Difference Between Stack and Heap: 37082321757012 bytes (0x21b9e69da354)
wagleayden@Ayden:~/os-labs/LAB 04/task2$
```

Figure 10: Run GCC compilation of lab2.c

The program prints the address of each variable using %p. The output shows the address of the Data, BSS, Stack, and Heap variables.

3) Identify which Segment has the Highest Addresses:

The Stack segment has the highest address. local_var is at 0x7fff2c3f9364.

4) Identify which Segment has the Lowest Addresses:

The Data and BSS segments have the lowest addresses. global_init (Data) is at 0x5e45123bb010 and global_uninit (BSS) is at 0x5e45123bb018. They are next to each other.

5) Calculate the Address Difference between Stack and Heap:

The stack address is 0x7fff2c3f9364 and the heap address is 0x5e4545a1f010. The difference between them is 37082321757012 bytes (0x21b9e69da354), as printed by the program.

In my output, the stack has the highest address (0x7fff2c3f9364), the heap is in the middle (0x5e4545a1f010), and the Data (0x5e45123bb010) and BSS (0x5e45123bb018) segments have the lowest addresses and . This matches [Figure 10](#). The addresses start with 0x5e45... instead of 0x404... because of ASLR, but the order of the segments is the same.

KNOWLEDGE CHECK QUESTIONS

Question 1

What is a data type in the context of programming and operating systems?

A data type defines what kind of value a variable can hold and how much memory it needs. It is an agreement between the programmer, the compiler, and the operating system.

It decides three things: how many bytes are allocated for the variable, how those bytes are read (an integer, a character, or a floating-point number), and which operations are allowed on it.

Example:

An int takes **4 bytes** on my system and stores whole numbers, while a char takes **1 byte** and stores a single character. When a variable is declared, the operating system uses its data type to decide how much memory to give it.

Question 2

Name the five main memory segments in a process memory layout.

The five main memory segments in a process memory layout are:

1. **Text:** Stores the program's machine code.
2. **Data:** Stores global and static variables that have an initial value.
3. **BSS:** Stores global and static variables that have no initial value.
4. **Heap:** Stores memory you allocate while the program runs using malloc. It grows upward.
5. **Stack:** Stores local variables and function calls. It grows downward.

Question 3

What is the difference between the Data segment and the BSS segment

The Data segment stores global and static variables that have an initial value.

Example: `int global_var = 150;`

The BSS segment stores global and static variables that have no initial value. The system sets BSS variables to zero when the program starts.

Example: `int bss_var;`

The OS sets BSS variables to zero when the program starts. BSS does not need to store values in the program file, so the file is smaller.

Question 4

Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

The text segment has a fixed size so it is placed at low addresses. The stack starts at the top and grows down, and the heap starts in the middle and grows up. This way the stack and heap have a lot of free space between them and can grow without hitting each other or the fixed segments.

Question 5

What happens to stack memory when a function returns? Who reclaims it?

When a function returns, its stack frame is popped and the stack pointer moves back. This happens automatically so the programmer does not need to free it. The space is then reused by the next function call.

Question 6

How does the OS know which memory belongs to which process?

Each process has its own virtual address space and its own page table. They are both managed by the OS. The page table maps the process's virtual addresses to physical memory so a process can only reach its own memory.

CONCLUSION

In this lab, data types and memory segments can be seen working on a real system. In Lab 1, the sizes matched the 64-bit column. Both long and the pointer were 8 bytes, which shows that my system is 64-bit. In Lab 2, the stack had the highest address and Data and BSS segments had the lowest addresses. The heap was in the middle. The difference between the stack and heap addresses was 37082321757012 bytes. This matches the memory layout shown in [Table](#) of the lab manual. The addresses change every time the program runs because of Address Space Layout Randomization (ASLR), but the order of the segments stays the same.

Overall, this lab helped me understand how the operating system organizes memory and why choosing the right data type matters.

GITHUB REPOSITORY

[GitHub Repository](#)